

CanlıFal — Flutter ⇌ Backend Entegrasyon & Performans Prompt'u

Bu doküman, Flutter uygulamasının CanlıFal backend'i ile **birebir uyumlu, tutarlı** ve **hızlı** çalışması için gereken tüm sözleşmeyi (contract) içerir.

Bir yapay zeka kod asistanına (Cursor, Copilot vb.) **sistem prompt'u** olarak verilebilir veya geliştirici referansı olarak kullanılabilir.

Makine-okunur tam sözleşme: `backend-docs/openapi.json` (Swagger), `backend-docs/postman_collection.json`, `backend-docs/endpoints_index.json`, `backend-docs/ENDPOINTS.md`.

Canlı sözleşme ayrıca MCP sunucusu (`mcp-server/`) üzerinden anlık okunabilir — bu sayede doküman asla eskimiez.

0. AI Asistanına Talimat (bu bölümü aynen uygula)

Sen CanlıFal Flutter uygulamasını geliştiren kıdemli bir mobil mühendissin. Aşağıdaki kurallar KESİNDİR ve backend ile %100 tutarlılık için zorunludur:

- TÜM API çağrıları tek bir Base URL ve merkezi bir Dio istemcisi üzerinden yapılır.
- Endpoint yolları için `/api/v1/...` öneki kullanılır (versiyonlu). Eski `/api/...` da çalışır ama yeni kodda DAİMA `/api/v1` kullan.
- Backend'in döndürdüğü standart zarf (envelope) yapısına ve hata kodlarına birebir uy.
- Kimlik doğrulama Bearer JWT (`accessToken`) iledir; 401 gelince refresh token ile yenile.
- Sürekli HTTP polling YAPMA. Gerçek zamanlı olaylar için SSE (Server-Sent Events) dinle.
- Medya (PNG/SVG/GIF/WEBP/MP4) doğrudan CDN URL'sinden gösterilir; API sadece URL döner.
- Backend'de bir alan eklenirse Flutter modelini kırma — bilinmeyen alanları yok say, eksik alanlar için null-safe ol.

1. Base URL & Versiyonlama

Ortam	Base URL
Production	<code>https://canlifal.com</code>
API öneki (versiyonlu)	<code>https://canlifal.com/api/v1</code>
API öneki (eski, geriye uyumlu)	<code>https://canlifal.com/api</code>

- `/api/v1/...` ile `/api/...` **aynı** endpoint'lere gider, **aynı** JSON'u döner. Yeni Flutter kodunda `/api/v1` kullan.
- Versiyonlu isteklerde yanıt `x-api-version: v1` başlığı taşır.

- **Tek Base URL** ilkesi: uygulamada base URL tek bir sabit/config değerinde tutulmalı (örn. `ApiConfig.baseUrl`).

```
class ApiConfig {
    static const String baseUrl = 'https://canlifal.com';
    static const String apiPrefix = '/api/v1';
    static String url(String path) => '$baseUrl$apiPrefix$path';
}
```

2. Standart Yanıt Zarfı (Envelope)

Modern endpoint'ler `lib/api-response.ts` üzerinden şu zarfı döner:

Başarılı:

```
{
  "success": true,
  "message": "Başarılı",
  "data": { },
  "pagination": { "page": 1, "limit": 30, "total": 120, "totalPages": 4, "hasNext": true, "hasPrev": false },
  "timestamp": "2026-08-01T00:00:00.000Z",
  "requestId": "uuid"
}
```

Hatalı:

```
{
  "success": false,
  "error": { "code": "VALIDATION_ERROR", "message": "Açıklama", "details": [] },
  "timestamp": "2026-08-01T00:00:00.000Z",
  "requestId": "uuid"
}
```

Not: Bazı eski/özel endpoint'ler doğrudan `{ success, data }` veya ham nesne döndürebilir (ör. mobil-login `{ accessToken, refreshToken, user }`, bazı listeler `{ streams, items, pagination }`). Bu yüzden parse ederken **savunmacı** ol: önce `data` anahtarını dene, yoksa gövdeyi doğrudan kullan.

```
dynamic unwrap(Map<String, dynamic> body) {
  if (body.containsKey('data')) return body['data'];
  return body; // eski/özel format
}
```

Standart Hata Kodları (backend ErrorCodes)

Kod	HTTP	Anlamı
UNAUTHORIZED	401	Oturum yok / token geçersiz
FORBIDDEN	403	Yetki yok
TOKEN_EXPIRED	401	Access token süresi doldu → refresh et
INVALID_TOKEN	401	Token bozuk
VALIDATION_ERROR	400	Girdi doğrulama hatası (details[] alanlarına bak)
MISSING_FIELD	400	Zorunlu alan eksik
NOT_FOUND	404	Kayıt yok
ALREADY_EXISTS / CONFLICT	409	Çakışma
INSUFFICIENT_CREDITS	402	Yetersiz kredi
INSUFFICIENT_JETONS	402	Yetersiz jeton
RATE_LIMITED	429	Çok fazla istek
SESSION_EXPIRED	400	Oturum süresi bitti
FEATURE_DISABLED	400	Özellik kapalı
INTERNAL_ERROR	500	Sunucu hatası
SERVICE_UNAVAILABLE	503	Servis kullanılamıyor
EXTERNAL_SERVICE_ERROR	502	Harici servis (ör. AI) hatası

Flutter tarafında bu kodları tek bir `enum ApiErrorCode` ile eşle ve UI davranışını koda göre belirle (ör. `INSUFFICIENT_JETONS` → jeton satın alma sayfasına yönlendir).

3. Kimlik Doğrulama (JWT) — Merkezi

Backend **dual auth** kullanır: mobil için `Authorization: Bearer <accessToken>`, web için oturum çerezi. Flutter **her zaman Bearer JWT** kullanır.

Token uçları

İşlem	Endpoint	Body	Döner
Kayıt	POST /api/v1/auth/mobile-register	{ name, email, password }	{ accessToken, refreshToken, user }
Giriş	POST /api/v1/auth/mobile-login	{ email veya username, password }	{ accessToken, refreshToken, user }
Token yenile	POST /api/v1/auth/mobile-refresh	{ refreshToken }	{ accessToken, refreshToken }
Google	POST /api/v1/auth/mobile-google	{ idToken }	{ accessToken, refreshToken, user }
Apple	POST /api/v1/auth/mobile-apple	{ identityToken, ... }	{ accessToken, refreshToken, user }
TikTok	POST /api/v1/auth/mobile-tiktok	sağlayıcıya göre	{ accessToken, refreshToken, user }
Çıkış	POST /api/v1/auth/logout	—	{ success }

- **accessToken** ömrü: 7 gün • **refreshToken** ömrü: 30 gün.
- Access token'ı flutter_secure_storage içinde sakla. Bellekte de tut, her istekte header'a ekle.
- 401 / TOKEN_EXPIRED gelince **bir kez** refresh dene; refresh de başarısızsa çıkışa yönlendir.

Dio Interceptor (zorunlu kalıp)

```
final dio = Dio(BaseOptions(
  baseUrl: ApiConfig.baseUrl + ApiConfig.apiPrefix,
  connectTimeout: const Duration(seconds: 10),
  receiveTimeout: const Duration(seconds: 15),
  headers: {'Accept': 'application/json'},
));

dio.interceptors.add(InterceptorsWrapper(
  onRequest: (options, handler) async {
    final token = await tokenStore.access;
    if (token != null) options.headers['Authorization'] = 'Bearer $token';
    handler.next(options);
  },
  onError: (e, handler) async {
    if (e.response?.statusCode == 401 && !_isRefreshing) {
      final ok = await _refreshToken(); // /auth/mobile-refresh
      if (ok) return handler.resolve(await _retry(e.requestOptions));
    }
    handler.next(e);
  },
));
```

4. Gerçek Zamanlı Olaylar — SSE (POLLING YAPMA)

Aşağıdaki olaylar **HTTP ile sürekli sorgulanmamalı**; backend bunları SSE ile anlık iletir. Her SSE ucu `text/event-stream` döner ve `emit*Event` fonksiyonlarıyla beslenir.

Alan	SSE Endpoint	Örnek olaylar
Canlı fal odası	<code>GET /api/v1/room/[sessionId]/stream</code>	<code>message</code> , <code>session_ended</code> , <code>session_extended</code>
Sohbet odası	<code>GET /api/v1/chat/rooms/[roomId]/stream</code>	<code>message</code> , <code>gift</code> , <code>koltuk/kullanıcı değişimi</code> , <code>DJ olayları</code>
Video yayını	<code>GET /api/v1/video-streams/[streamId]/stream</code>	<code>streamMessage</code> , <code>gift</code> , <code>streamEnded</code> , <code>PK olayları</code>
Falcı oturum istekleri	<code>GET /api/v1/fortune-tellers/sessions/stream</code>	<code>session_request</code> , <code>session_cancelled</code>
Bildirimler	<code>GET /api/v1/notifications/stream</code>	yeni bildirim

Kapsanan olaylar (talebe göre): hediye gönderimi, mesajlar, koltuk değişiklikleri, kullanıcı giriş/çıkışı, PK davetleri, canlı yayın başlangıcı, fal isteği — **hepsi SSE ile**.

Flutter SSE tüketimi

- Paket: `flutter_client_sse` veya `dio` ile `responseType: ResponseType.stream`.
- SSE bağlantısına `Authorization` header'ı ekle (JWT).
- Bağlantı koparsa **exponential backoff** ile yeniden bağlan (1s, 2s, 4s... max 30s).
- Uygulama arka plana geçince SSE'yi kapat, öne gelince yeniden aç.
- SSE'yi **tek doğruluk kaynağı** yap; ekranı state (Riverpod/Bloc) üzerinden güncelle, tekrar GET atma.
- WebRTC sinyalleşmesi (canlı görüşme) için `/api/v1/room/signal` (GET bekleyen sinyaller, POST gönder, DELETE eski sinyalleri temizle) kullanılır.

```
SSEClient.subscribeToSSE(
  method: SSERequestType.GET,
  url: ApiConfig.url('/video-streams/$streamId/stream'),
  header: {'Authorization': 'Bearer $token', 'Accept': 'text/event-stream'},
).listen((event) {
  final type = event.event; // 'gift', 'streamMessage', 'streamEnded'...
  final data = jsonDecode(event.data ?? '{}');
  eventBus.dispatch(type, data);
});
```

5. Hediye Sistemi & Medya (PNG/SVG/GIF/WEBP/MP4)

Hediye uçları (`/api/v1/gifts/types` , `/api/v1/gifts/catalog` , `/api/v1/video-streams/gifts` , `/api/v1/live/gift-types` , `/api/v1/chat/rooms/[roomId]/gifts`) her hediye için **birleşik medya alanları** döner:

Alan	Açıklama
<code>type</code>	Hediye türü
<code>mediaType</code>	"video" "image" "gif" "lottie" "svga"
<code>assetFormat</code>	png/jpeg/webp/gif/svg/svga/lottie/mp4/webm
<code>fileUrl</code>	Ana medya dosyasının tam CDN URL'si
<code>thumbnailUrl</code>	Poster/kapak görseli (video için otomatik üretilir)
<code>previewUrl</code>	Küçük önizleme görseli
<code>width</code> , <code>height</code>	Piksel boyutları (varsa)
<code>duration</code>	Video süresi ms (varsa)
<code>mimeType</code>	Örn. "video/mp4"

Flutter render kuralları

- `mediaType == 'video'` → `video_player` ile: `autoplay + muted + loop` , `BoxFit.contain` , `poster = thumbnailUrl` . **Küçük kare gösterme.**
- `mediaType == 'image'/'gif'/'webp'` → `CachedNetworkImage(fileUrl)` .
- `svga` → `SVGAPlayer` , `lottie` → `lottie` paketi.
- Bilinmeyen tür → `previewUrl` / `thumbnailUrl` ile statik görsel fallback.
- Tüm medya doğrudan CDN'den gelir**; ayrıca proxy/indirme yapma. API sadece URL döner.

6. Cache, Sayfalama & Performans (Flutter tarafı)

Sunucu cache'i (bilgi)

Backend şu verileri önbellekler ve `Cache-Control` başlığı döner: hediye listesi, ödeme yöntemleri, kredi paketleri, profil, rozetler, emoji, ayarlar. Flutter bu başlıklara saygı gösterir.

Flutter zorunlu performans kalıpları

- Dio + HTTP cache:** `dio_cache_interceptor` ile GET yanıtlarını `Cache-Control` / `ETag` 'e göre önbelleğe al (304 desteği var).
- Görsel/video cache:** `cached_network_image` + `flutter_cache_manager` .

3. **Sayfalama:** liste uçları `?page=&limit=` alır ve `pagination` döner. Sonsuz kaydırma (infinite scroll) kullan, tüm listeyi tek seferde çekme. `limit max 100`.
4. **Paralel istekler:** açılışta bağımsız çağrılar `Future.wait([...])` ile paralel yap.
5. **Lazy loading:** liste öğelerini `ListView.builder` ile tembel yükle.
6. **State yönetimi:** Riverpod veya Bloc; gereksiz `rebuild` leri `select / const` widget ile engelle.
7. **Offline:** son başarılı yanıtları yerelde tut (Hive/Isar), ağ yokken göster.
8. **Retry:** geçici hatalarda (`RATE_LIMITED` , 5xx, timeout) exponential backoff ile 2-3 deneme.
9. **ETag/If-None-Match:** cache interceptor otomatik gönderir → 304 ile bant genişliği tasarrufu.

Hız hedefleri (referans)

- Uygulama açılışı: 1-2 sn · Ana sayfa: <1 sn · Canlı oda girişi: <2 sn · Hediye görünmesi: 100-300 ms · PK daveti: <500 ms · Mesaj gecikmesi: <100 ms (SSE ile).
- **Not:** AI üreten uçlar (burç/fal yorumu) doğaları gereği birkaç saniye sürer — bunları anlık hedeflere sokma, kullanıcıya “hazırlanıyor” durumu göster.

7. Sözleşme Senkronizasyonu (Web ⇌ Flutter Tutarlılığı)

- Web ve Flutter **aynı endpoint’leri, aynı JSON’u, aynı hata kodlarını, aynı JWT mekanizmasını ve aynı SSE olay adlarını** kullanır.
- Backend’de bir endpoint değişince sözleşme dokümanları şu 3 script ile yeniden üretilir:
 1. `scripts_docs/generate_api_docs.py` → `endpoints_index.json` , ilk `openapi.json`
 2. `scripts_docs/build_openapi_postman.py` → `openapi.json` , `postman_collection.json`
 3. `scripts_docs/build_endpoint_list.py` → `ENDPOINTS.md`
- MCP sunucusu (`mcp-server/`) bu dosyaları ve canlı route kaynaklarını **anlık** okur; Cursor gibi araçlar Flutter’ı her zaman güncel sözleşmeye göre yazar.
- **Kural:** Flutter modelleri `fromJson` ‘da bilinmeyen alanları yok saymalı, eksik alanlarda null-safe olmalı ki backend’e alan eklenince mobil kırılmasın.

Öneri: OpenAPI’den model üretimi

`openapi.json` ‘dan Dart modelleri üretmek için `openapi-generator` (dart-dio) veya `swagger_dart_code_generator` kullan. Bu, elle model yazımını ve tutarsızlığı ortadan kaldırır.

8. Sık Kullanılan Endpoint'ler (hızlı başlangıç)

Amaç	Endpoint
Mobil config	GET /api/v1/mobile/config
Ana sayfa verisi	GET /api/v1/mobile/home
Fal menüsü	GET /api/v1/mobile/fortune-menu
Kullanıcı profili	GET /api/v1/user/profile
Başka kullanıcı profili	GET /api/v1/mobile/user-profile/[userId]
Aktif oturumlar	GET /api/v1/user/active-sessions
Canlı yayın listesi	GET /api/v1/video-streams?page=1&limit=30
Yayın detayı	GET /api/v1/video-streams/[streamId]
Sohbet odaları	GET /api/v1/chat/rooms
Hediye türleri	GET /api/v1/gifts/types
Kredi paketleri	GET /api/v1/credit-packages
Ödeme yöntemleri	GET /api/v1/payment-methods
Günlük burç	GET /api/v1/horoscope/daily
Bildirim akışı (SSE)	GET /api/v1/notifications/stream

Tam liste (473 benzersiz yol / 736 handler): [backend-docs/ENDPOINTS.md](#) . Her endpoint'in gövde alanları ve auth türü orada işaretlidir (🔄 Dual / 🌐 Oturum / 🌍 Public, 🔒 ADMIN).

9. Yapılacaklar Kontrol Listesi (Flutter)

- [] Tek `ApiConfig.baseUrl` + `/api/v1` öneki.
- [] Merkezi Dio istemcisi + JWT interceptor + 401 refresh.
- [] Standart zarf/hata kodu parse (`unwrap` + `ApiErrorCode`).
- [] `flutter_secure_storage` ile token saklama.
- [] SSE dinleyicileri (oda, yayın, sohbet, bildirim, falcı) + backoff reconnect.
- [] Hediye render (video: autoplay/muted/loop/contain/poster).
- [] Dio cache + görsel/video cache + ETag/304.
- [] Sayfalama + infinite scroll + `Future.wait` paralel açılış.
- [] Riverpod/Bloc + gereksiz rebuild engelleme.
- [] Offline cache (Hive/Isar) + retry/backoff.

- [] OpenAPI'den model üretimi (tutarlılık).

Bu doküman canlı backend sözleşmesinden üretilmiştir. Şüphede kalınca `backend-docs/openapi.json` ve `ENDPOINTS.md` esas alınır; en güncel hali için MCP sunucusu sorgulanır.